

Modelos Locales e Inferencia

Docente:

Mg. Ing Abraham Rodriguez - FIUBA

Programa de la materia

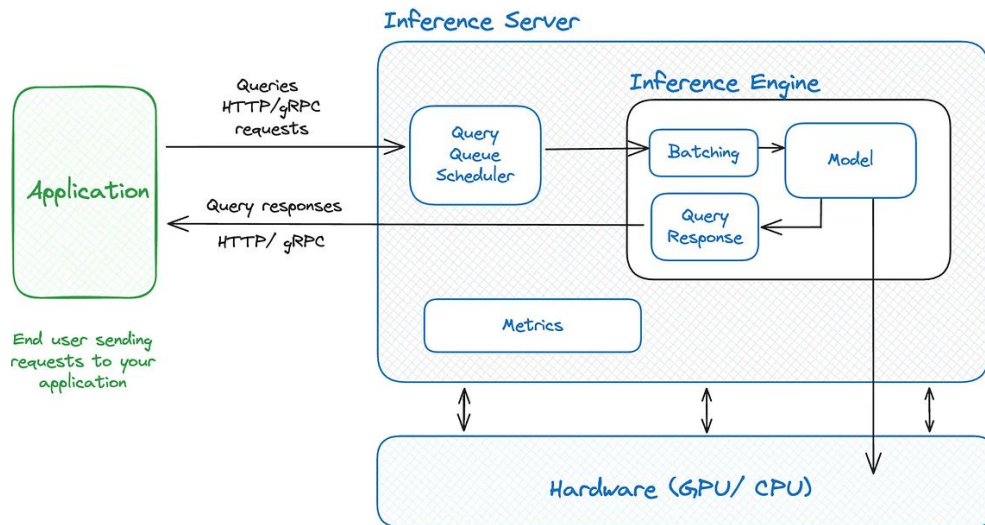
1. Repaso de Transformers, Arquitectura y Tokenizers.
2. Arquitecturas de LLMs, Transformer Decoder.
3. MoEs, técnicas de prompts, intro evaluación de LLMs.
4. Ecosistema actual, APIs, costos, HuggingFace y OpenAI.
5. **Modelos locales, servidores de inferencia, Reinforcement Learning.**
6. Modelos de Embeddings, vector DBs, naïve RAG, chatbots.
7. Intro a Agentes, LLMs de razonamiento.
8. Multimodalidad y estado del arte.
- 9.

Inference Servers

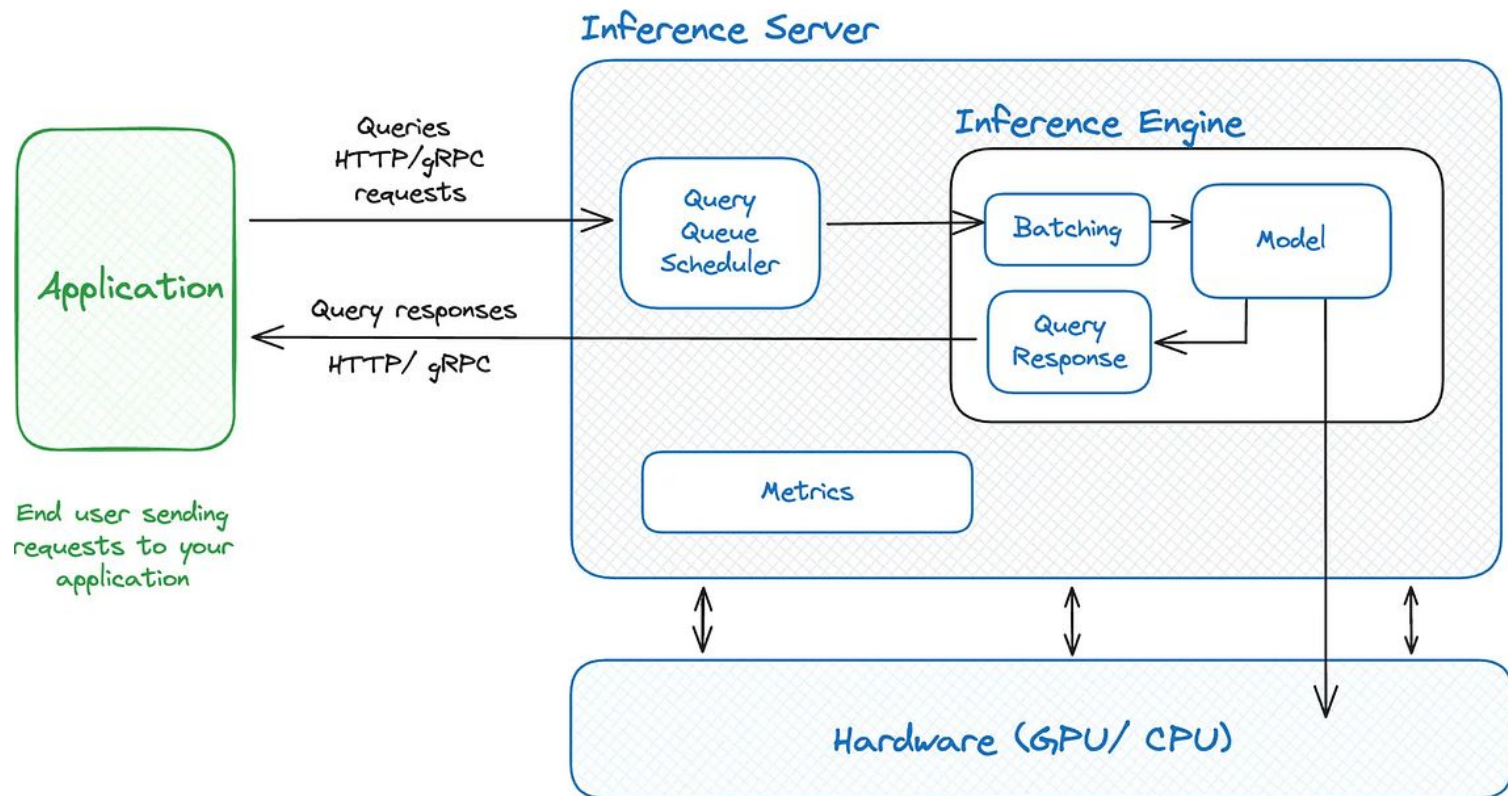
Una LLM **preentrenada** debe ser puesta y servida en modo inferencia para su aplicación, normalmente como *modelo de chat de tipo instruct* para realizar tareas mediante prompt/context engineering como:

- Agente conversacional
- Sistemas RAG
- Generación de texto
- Análisis de documentos

Inference Server Benchmarks



Inference Servers



Inference Servers

Los inference server suelen abstraer varios pasos para poder servir a una LLM siendo:

- Descarga y manejo de modelos en formato eficiente (Safetensors, GGUF).
- Conversión de modelos.
- Quantization.
- El uso de mejoras como FlashAttention.
- Optimización de capas.
- Detección y selección de hardware.
- Algoritmos eficientes de decodificación, manejo cache, etc.

[Ejemplo conversión](#)

[Lectura safetensors](#)

[Ejemplo GGML](#)

[Implementación de Inference Server](#)

[Ollama Engine](#)

[Ollama Quantization](#)

Inference Servers

En LLM providers closed-source (OpenAI, Anthropic, etc) es poco común que expongan algoritmos de decodificación o manipulación de parámetros del inference server, mas alla de temperatura, topK, topP, etc.

En el caso de [vLLM](#), decodificación especulativa está disponible.

[OpenAI Chat Completion](#)

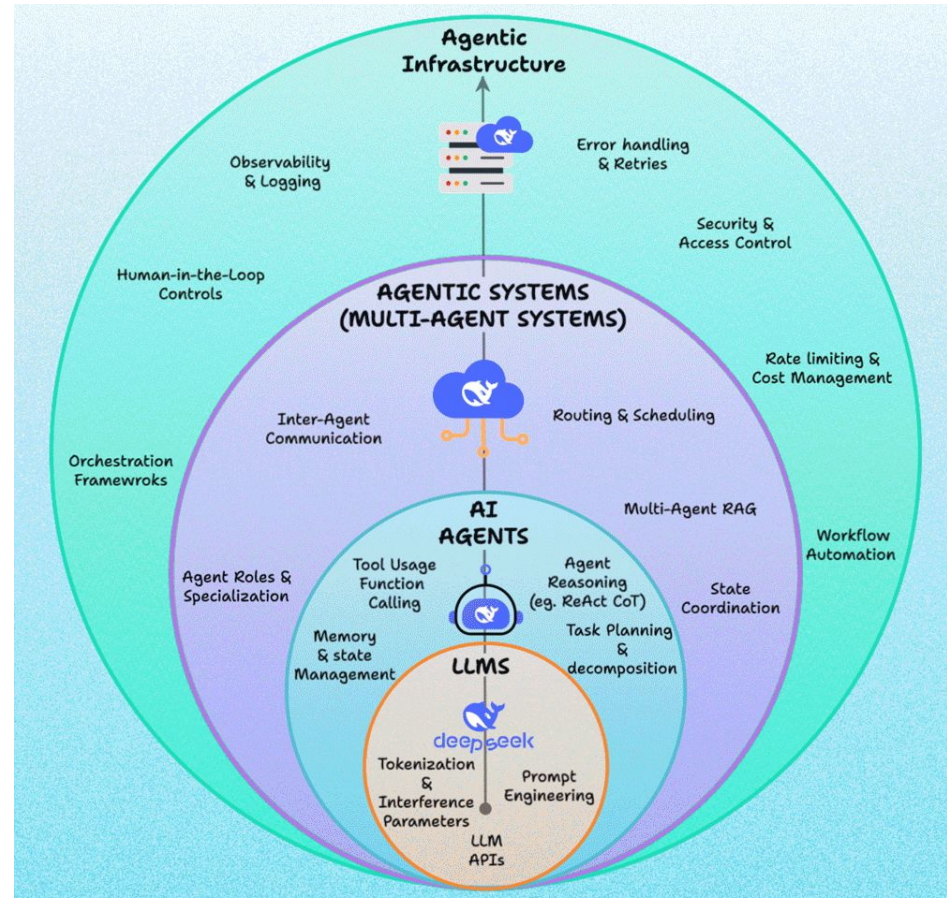
[Claude Messages](#)

[Amazon Rufus Speculative Decoding](#)

Inference Servers

- Los sistemas que utilizan LLMs funcionan bien en prototipos, pero escalar a producción implica **desafíos de costo, latencia, throughput, mantenimiento, desarrollo, integraciones y otros.**
- Escalabilidad debe considerarse en 3 niveles:
 1. **Modelos (LLMs)**
 2. **Agentes**
 3. **Infraestructura**
 4. **Arquitectura de sistemas**

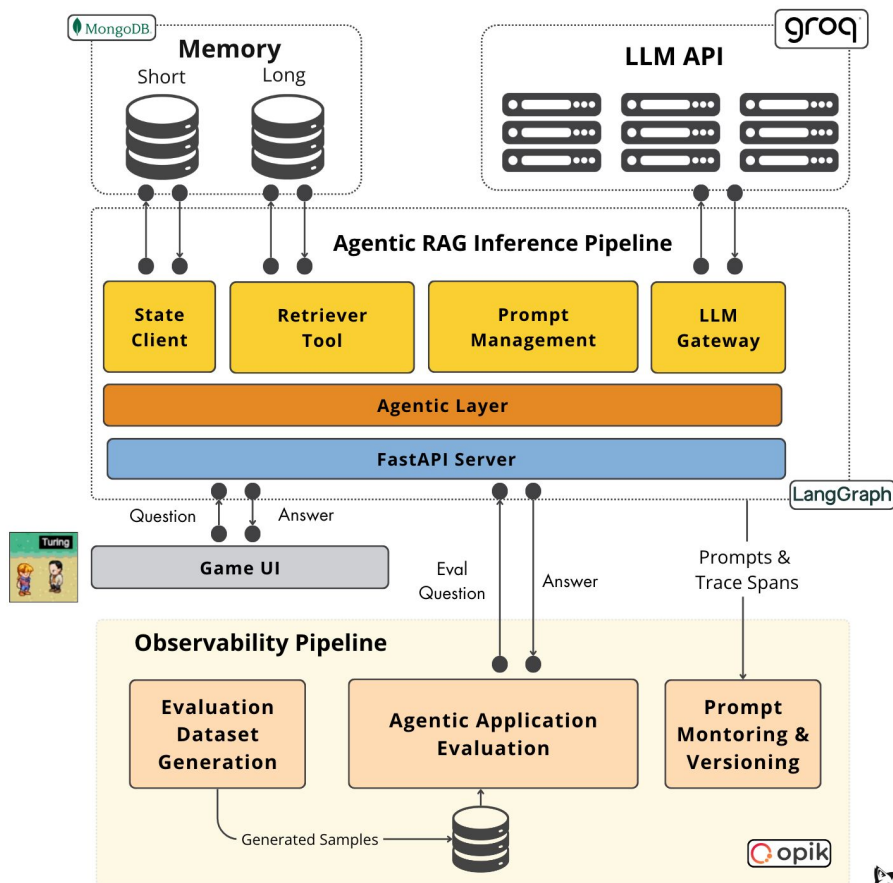
Siendo el más importante el LLM



Inference Servers

El inference server es una capa que comúnmente no se expone en la práctica, se considera una capa de bajo nivel con respecto a aplicaciones de LLM, una caja negra.

Sin embargo es la capa más crítica con respecto a un sistema de agentes o AI moderno.



Algunos Inference Servers

Llama.cpp

[Llama.cpp](#) es un proyecto en C/C++ cuyo objetivo principal es permitir la inferencia de LLMs y modelos multimodales con configuración mínima y alto rendimiento en una amplia variedad de hardware, tanto **localmente como en la nube**. Consiste en:

- Implementación en C/C++ puro sin dependencias.
- Apple Silicon, optimizado mediante ARM NEON, y los frameworks Accelerate y Metal.
- Soporte para AVX, AVX2, AVX512 y AMX en arquitecturas x86.
- Quantización de 1.5 bits, 2 bits, 3 bits, 4 bits, 5 bits, 6 bits y 8 bits para una inferencia más rápida y menor uso de memoria.
- Kernels CUDA personalizados para ejecutar LLMs en GPUs de NVIDIA
- Soporta GPUs AMD mediante HIP y GPUs MTT de Moore Threads mediante MUSA).
- Soporte para backend Vulkan y SYCL.
- Inferencia híbrida CPU+GPU para acelerar parcialmente modelos que superen la capacidad total de VRAM.

Llama.cpp

El proyecto expone una [REST API](#) compatible con el formato OpenAI.

Existe un proyecto llamado [Llama-cpp-python](#) que expone bindings a Python.

[Langchain Docs](#)

[LlamaIndex Docs](#)

[Build on Android using Termux](#)

[Running Llama-2 on AWS with Llama.cpp](#)

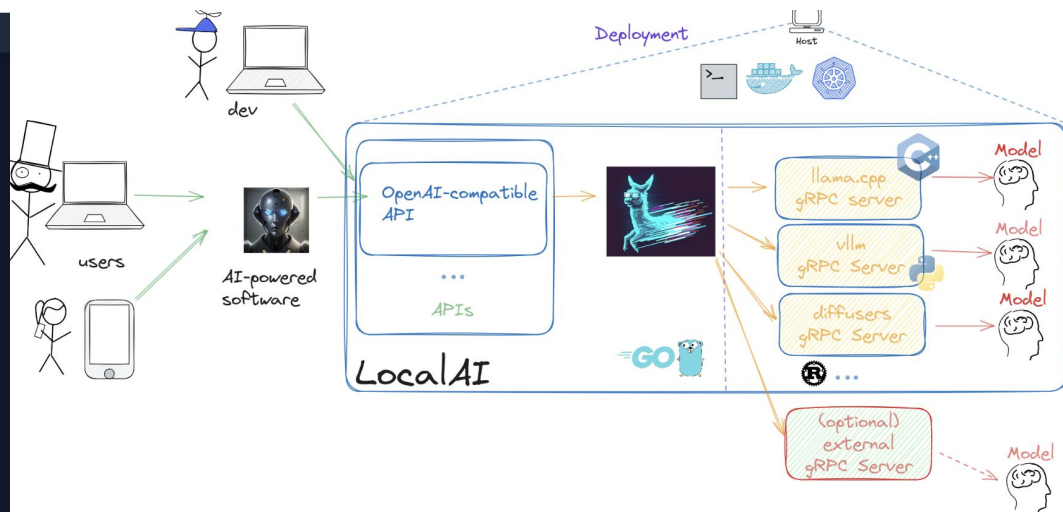
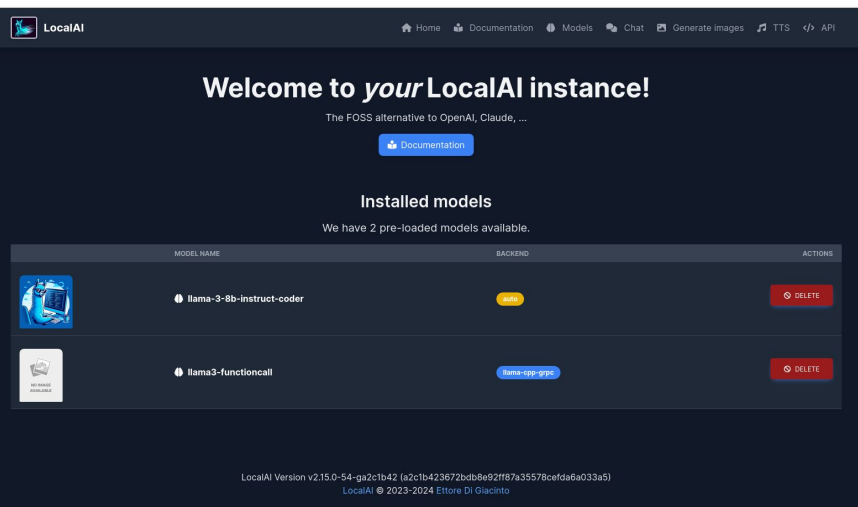
Llama.cpp LM Studio

LM Studio es un Wrapper de Llama.cpp provee una GUI para ejecutar modelos locales, permite realizar RAGs locales al agregar documentos.

LocalAI

LocalAI es un proyecto open-source hecho en Go que implementa múltiples plataformas como backend siendo: Llama.cpp, vLLM, entre otras, el objetivo es ofrecer una REST API, GUI y una suite de modelos para ser utilizados de manera gratuita y privada.

Es capaz de integrar modelos de texto, multimodales y diffusion.



Ollama

[Ollama](#) es un proyecto open-source realizado en [Go](#) que consiste en un **wrapper de Llama.cpp** y enfatiza el uso de LLMs de **manera local** mediante un CLI simple, por ejemplo el comando en terminal descarga y optimiza el modelo [Mistral-7B](#) desde huggingface:

```
- ollama run mistral
```

Ollama expone una [REST API](#) propia y una [experimental](#) con el formato de OpenAI.

Compatible con [GPUs NVIDIA y AMD](#)

Langchain provee [documentación](#) para integrar [Ollama](#).

[Scaling Local Inference to the Cloud](#)

[How to Deploy local LLM using Ollama Server and Ollama Web UI on Amazon EC2](#)

[Running Ollama on Raspberry Pi](#)



vLLM

[vLLM](#) es un proyecto hecho en Python presentado en el paper “[Efficient Memory Management for Large Language Model Serving with PagedAttention](#)”, implementa:

- Gestión eficiente de la memoria de keys y values con PagedAttention.
- Quantization: GPTQ, AWQ, INT4, INT8 y FP8.
- Kernels CUDA optimizados, incluyendo integración con FlashAttention y FlashInfer.
- Chunked prefill
- Integración fluida con modelos populares de Hugging Face
- Diversos algoritmos de decodificación como parallel sampling y beam search.
- Salidas en streaming.
- REST API compatible con OpenAI.
- Soporte para GPUs NVIDIA, CPUs y GPUs AMD, CPUs Intel, TPU.
- Prefix caching.
- Soporte multi-LoRA.



vLLM Serving

vLLM está listo para servir en ambientes productivos, ofrece múltiples guías para configurarlo:

[Docker Deployment](#)

[Kubernetes Deployment](#)

[Deployment with Nginx Load Balancer](#)

[Production Metrics](#)

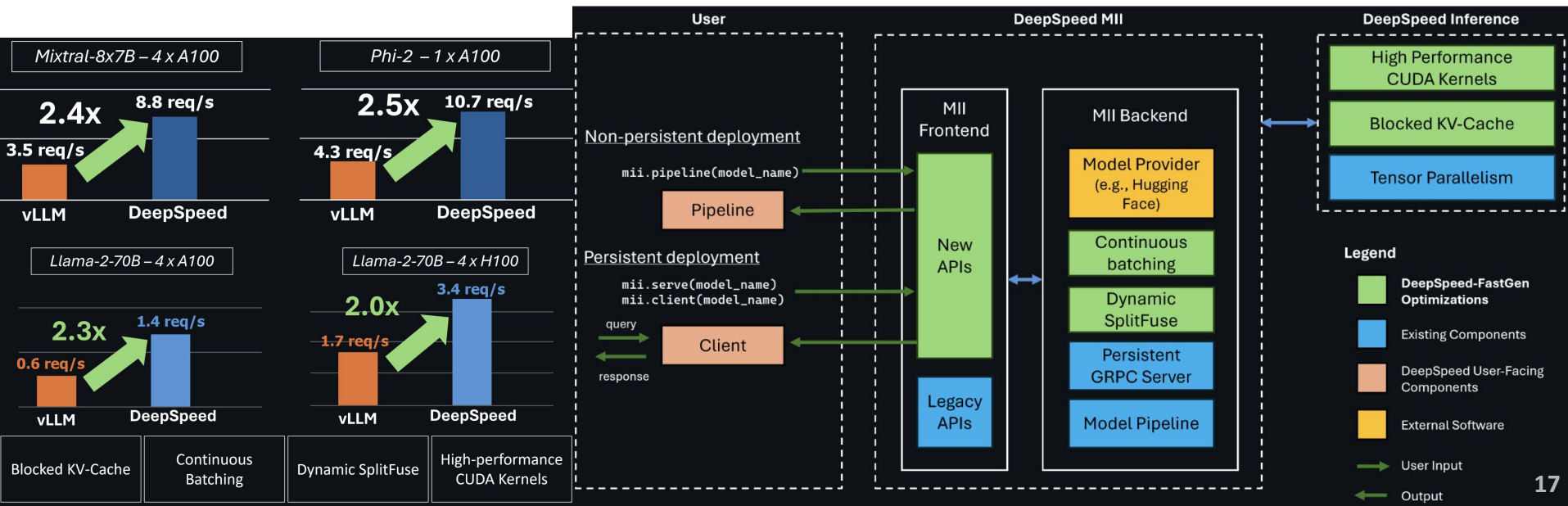
[Triton Deployment](#)

[BentoML Deployment](#)

[Supported Models](#)

DeepSpeed-MII

DeepSpeed-MII es un proyecto de Microsoft realizado en Python que promete ofrecer mayor rendimiento que vLLM, gracias a 4 Tecnologías. Esta basado en DeepSpeed un wrapper ligero de Pytorch que provee optimizaciones para LLMs.



NVIDIA Triton Inference Server

Triton, es un software open-source de la plataforma de NVIDIA que permite deploy cualquier modelo de múltiples frameworks como : TensorRT, Pytorch, ONNX, entre otros.

Ejecución concurrente de modelos

Pipelines de modelos usando Ensembling o Scripting de Lógica de Negocio (BLS)

APIs REST y gRPC.

Una API en C y una API en Java permiten la aplicación para casos de edge computing y otros usos en proceso.

Métricas y estadísticas del servidor y recursos.

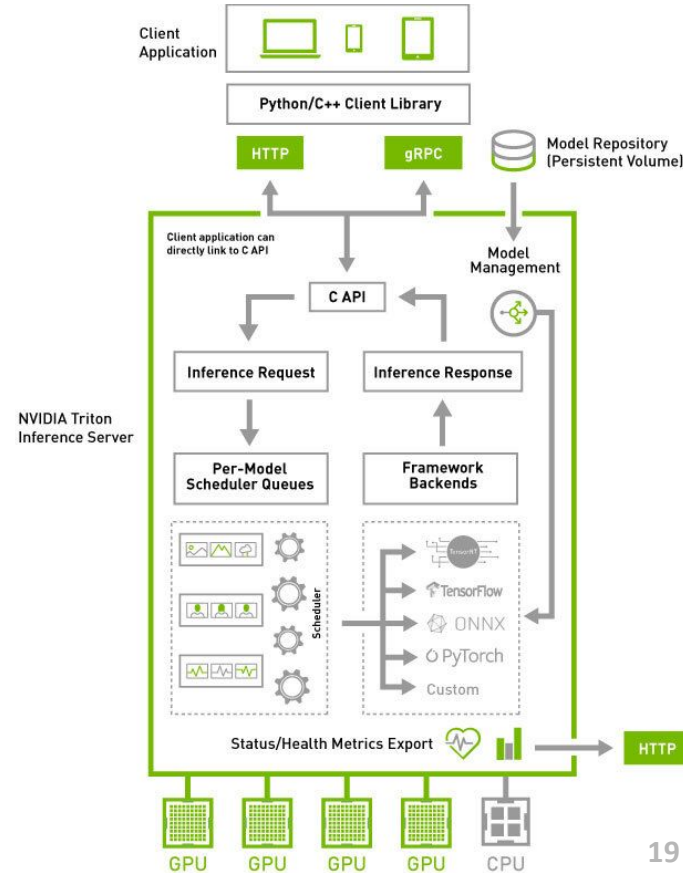
NVIDIA Triton Inference Server

[Deploying with Triton](#)

[Triton Container](#)

[Triton Tutorials](#)

[Kubernetes Deployment](#)



Jetson-inference

[NVIDIA Jetson](#) cuenta con una plataforma para realizar inferencia on Edge, para lograr configurar el sistema embebido se debe seguir la guía de [jetson-inference](#).

[Text-generation](#)

Gemma.cpp

[Gemma.cpp](#) es un proyecto open-source de Google, similar a Llama.cpp, sin embargo está focalizado en la familia de modelos Gemma.

Cuenta con [Python Bindings](#).

No expone una REST API.

ROCm (AMD)

AMD es soportado por varios inference servers como vLLM, para ello véase la [documentación](#).

[ROCm inference optimization](#).

[Making AMD GPUs competitive for LLM inference](#)

Google AI-Edge

<https://ai.google.dev/edge>

Transformers en el Edge

La mayoría de aplicaciones de transformers son basadas en la nube sin embargo, los modelos han evolucionado a la par del hardware haciendo capaz de ejecutar LLMs capaces en hardware modesto.

Las claves para ejecutar LLMs en dispositivos edge son:

- **Arquitectura de transformer ligera**
- **Inference server**
- **Aceleración por hardware**

[Edge Agentic AI Gemma 4](#)

[Gemini Robotics](#)

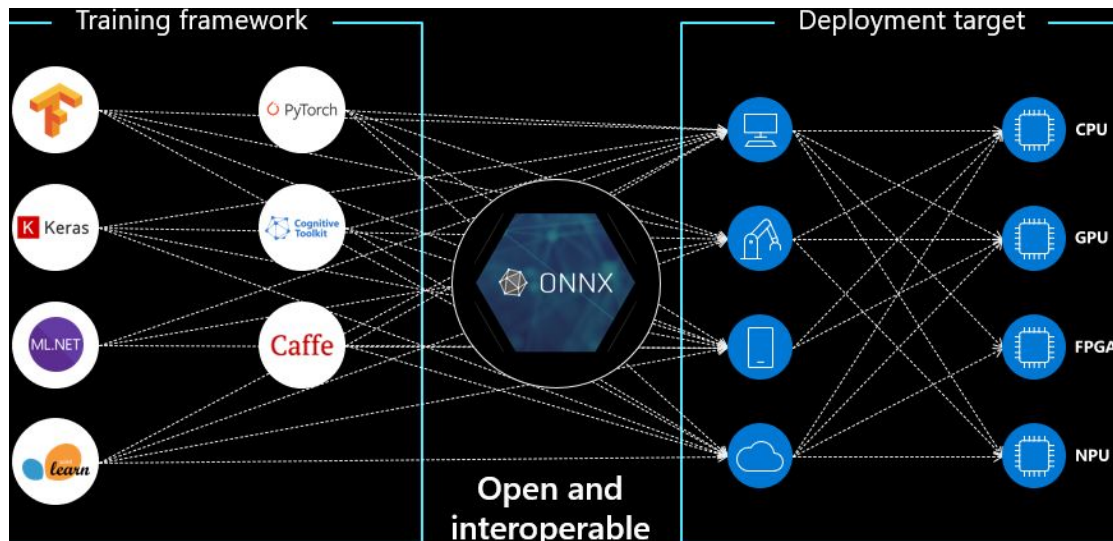
ONNX

ONNX

Open Neural Network Exchange

(ONNX) es un ecosistema open source el cual permite:

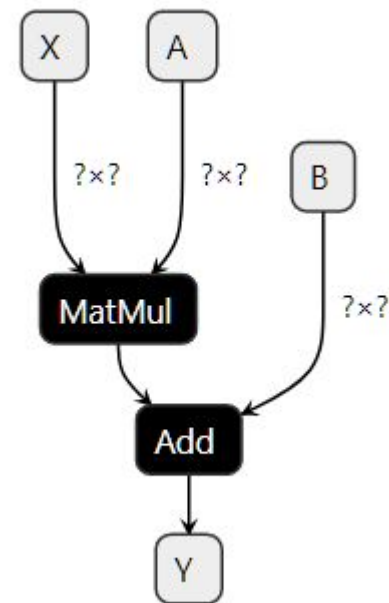
- **Representar** modelos de ML y DL de manera universal.
- **Interoperabilidad** entre herramientas, frameworks y ambientes.
- Define un **set de operadores/capas**.
- **Set de herramientas** de entrenamiento, inferencia, y visualización.



ONNX

El formato ONNX es similar a un lenguaje de programación **especializado en funciones matemáticas**. Define las operaciones necesarias que un modelo necesita para implementar una función de inferencia. Es **básicamente un grafo de cómputo**.

ONNX apunta a proveer un lenguaje común que cualquier framework de ML/DL pueda utilizar. Hacer mas facil el despliegue de modelos en ambientes productivos. Un **runtime ONNX** puede ser implementado y optimizado para la tarea de ejecucion de modelos.



ONNX Operadores y Tipos

ONNX ofrece la mayoría de operadores necesarias para implementar funciones de inferencia (Suma, MatMul, Conv, etc.)

ONNX utiliza tensores, los cuales son arrays multidimensionales definidos por:

- type: el tipo del elemento, el mismo tipo para todos los elementos de un tensor.
- shape: un array con todas las dimensiones, puede ser vacío, y la dimensión puede ser null.
- contiguous array: representa todos los valores.

```
1: onnx.TensorProto.FLOAT
2: onnx.TensorProto.UINT8
3: onnx.TensorProto.INT8
4: onnx.TensorProto.UINT16
5: onnx.TensorProto.INT16
6: onnx.TensorProto.INT32
7: onnx.TensorProto.INT64
8: onnx.TensorProto.STRING
9: onnx.TensorProto.BOOL
10: onnx.TensorProto.FLOAT16
11: onnx.TensorProto.DOUBLE
12: onnx.TensorProto.UINT32
13: onnx.TensorProto.UINT64
14: onnx.TensorProto.COMPLEX64
15: onnx.TensorProto.COMPLEX128
16: onnx.TensorProto.BFLOAT16
17: onnx.TensorProto.FLOAT8E4M3FN
18: onnx.TensorProto.FLOAT8E4M3FNUZ
19: onnx.TensorProto.FLOAT8E5M2
20: onnx.TensorProto.FLOAT8E5M2FNUZ
21: onnx.TensorProto.UINT4
22: onnx.TensorProto.INT4
23: onnx.TensorProto.FLOAT4E2M1
24: onnx.TensorProto.FLOAT8E8M0
25: onnx.TensorProto.UINT2
26: onnx.TensorProto.INT2
```

De Framework a ONNX

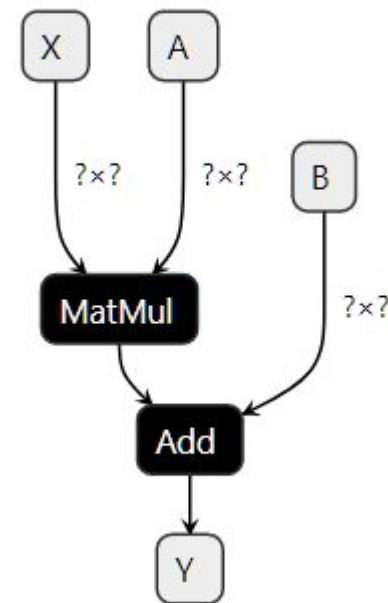
En entornos de producción (cloud, edge o sistemas locales), no es común ejecutar modelos directamente desde scripts de Python utilizando el mismo framework con el que fueron entrenados (TensorFlow, PyTorch, etc.).

En su lugar, los modelos se exportan a formatos de inferencia optimizados (como ONNX, TensorRT o TFLite) y se ejecutan en runtimes especializado

Los frameworks de entrenamiento están diseñados para flexibilidad y experimentación, no para máxima eficiencia.

En producción se prioriza:

- Baja latencia
- Alto throughput
- Uso eficiente de CPU/GPU/NPU



De Framework a ONNX

Los frameworks de entrenamiento están diseñados para flexibilidad y experimentación, no para máxima eficiencia.

En producción se prioriza:

1. Optimización para inferencia

- Baja latencia
- Alto throughput
- Uso eficiente de CPU/GPU/NPU

2. Separación entre entrenamiento e inferencia

Se desacoplan ambos procesos:

- **Entrenamiento:** dinámico, experimental, costoso
- **Inferencia:** estable, determinista, orientada a servicio

Esto facilita:

- Versionado independiente del modelo
- Despliegues más controlados
- Mayor reproducibilidad

De Framework a ONNX

3. Uso eficiente de recursos

Los frameworks de entrenamiento incluyen sobrecarga innecesaria en producción:

- Autograd (gradientes)
- Operaciones no optimizadas para inferencia
- Dependencias pesadas

Los runtimes de inferencia optimizan:

- Fusión de grafos
- Kernels especializados
- Reducción de memoria

4. Escalabilidad y robustez

Los sistemas de serving están diseñados para:

- Manejo de múltiples requests concurrentes
- Control de latencia bajo carga
- Batching dinámico
- Observabilidad y monitoreo

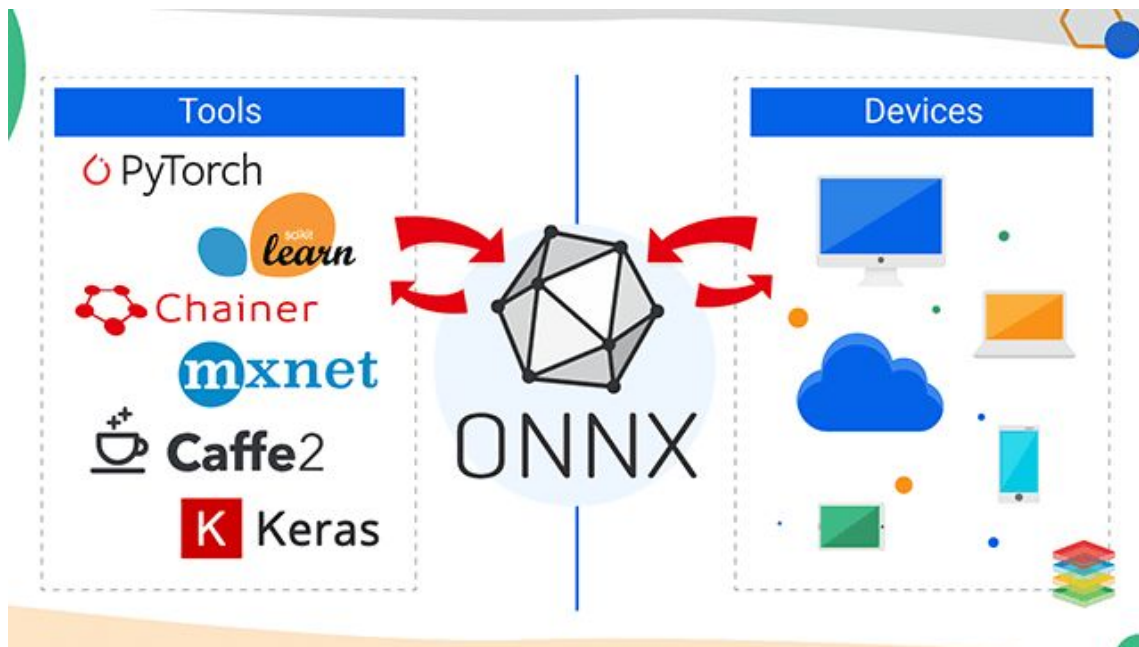
De Framework a ONNX

La mayoría de frameworks cuentan con **convertidores** a ONNX

[Pytorch ONNX](#)

[Tensorflow ONNX](#)

[Pytorch Export to ONNX](#)



ONNX Runtime

Teniendo un modelo en formato ONNX, el siguiente paso es **ejecutarlo** mediante un runtime compatible.

[ONNX Runtime](#) es un ambiente de ejecución multiplataforma y multi-lenguaje compatible con distintos hardwares, drivers y sistemas operativos utilizando aceleradores de hardware así como optimizaciones de grafos y transformaciones.

	Optimize Inferencing				Gen AI API			
Platform	Windows	Linux	Mac	Android	iOS	Web Browser		
API	Python	C++	C#	C	Java	JS	Obj-C	WinRT
Architecture	X64	X86	ARM64	ARM32	IBM Power			
Hardware Acceleration	Default CPU	CoreML	CUDA	DirectML	MIGraphX			
	NNAPI	oneDNN	OpenVINO	ROCm	QNN			
	Tensor RT	ACL (Preview)	Arm NN (Preview)	Azure (Preview)	CANN (Preview)			
	Rockchip NPU (Preview)	TVM (Preview)	Vitis AI (Preview)	XNNPACK (Preview)				
Installation Instructions	Please select a combination of resources							

Edge y ambientes

[Despliegue en dispositivos Edge e IoT](#)

[Qualcomm AI Hub](#)

[NXP ML](#)

Preguntas?